MEMORY-BANDWIDTH-AWARE MULTI-PRECISION ANYTIME REASONING FOR ON-DEVICE LARGE LANGUAGE MODELS

Anonymous authors
Paper under double-blind review

ABSTRACT

We present MP-CHAR, an inference-time controller that unlocks high-quality large-language-model reasoning on commodity GPUs by coordinating beam search, mixed-precision arithmetic, and a zero-copy multi-view key-value cache inside a single resource-aware loop. Laptop-class accelerators are throttled by the product of arithmetic load and memory traffic; existing compute-adaptive (CHAR) or memory-adaptive (ARaR) decoders address only one side of this bottleneck and stall once beam width exceeds eight. MP-CHAR introduces a three-tier precision pyramid (4-, 8-, and 16-bit) with in-layer promotion, lightweight assessor heads that estimate process reward, remaining FLOPs, remaining bytes, and uncertainty, and a bandwidth-aware A* controller whose heuristic jointly optimises accuracy, latency, and energy. All precisions share one QLoRA-packed weight tensor, eliminating duplication, and the hardware-specific cost weights are auto-calibrated in 100 ms at start-up. Without fine-tuning the 7-billion-parameter backbone, MP-CHAR cuts average energy per problem by 22 %, doubles solved-per-second at equal accuracy relative to FP16 CHAR, and fits within 10 GB of VRAM on an RTX-3080-laptop across BIG-Bench-Hard, GSM8K, MATH, and Easy2Hard-Bench. Ablations confirm the benefit of the 4-bit tier and of uncertainty-aware assessor heads, while micro-benchmarks show 40 % lower cache traffic and 14 % higher throughput once bandwidth terms are enabled. Code and trained assessor are open-sourced, providing the first template for precision-adaptive inference on consumer hardware.

1 INTRODUCTION

Large language models (LLMs) have started to migrate from data-centre servers to consumer laptops and edge devices, enabling offline tutoring tools, code assistants, and accessibility applications. Yet their decoding cost remains prohibitive: maintaining a beam width of sixteen on a 7-billion-parameter model saturates both tensor-core throughput and DRAM bandwidth on a typical laptop GPU. Compute-adaptive decoders such as CHAR reduce arithmetic load, while memory-adaptive siblings such as ARaR prune low-value branches into cheaper caches; each alleviates only half of the bottleneck, and throughput plateaus once beam width exceeds eight.

Modern accelerators (NVIDIA RTX-30/40, Apple M-series, Intel ARC) can switch among FP16/BF16, INT8, FP8, and INT4 execution paths in microseconds; nevertheless, current reasoning engines still choose a single precision for an entire forward pass. This wastes energy on easy branches and, conversely, denies difficult hypotheses the numerical fidelity they require. We therefore ask: Can a single beam-search graph dynamically schedule precision so that easy paths run in four bits while hard paths retain sixteen-bit accuracy—without duplicating weights or key-value (KV) caches and while respecting device-specific bandwidth limits?

Three challenges arise. (i) The controller must predict—early and cheaply—when a four-bit approximation would compromise the final answer. (ii) Precision changes must avoid extra memory traffic that cancels arithmetic gains. (iii) Cost weights depend on individual devices and even on their thermal state, so they must be measured online rather than hard-coded.

Below we highlight the core architectural features that allow MP-CHAR to address these challenges.

- **Precision pyramid** All hypotheses start in P4, top-k branches are promoted to P8, and ambiguous ones to P16—all from a single QLoRA tensor.
- **Layer-progressive assessors** After each block, lightweight heads output process reward, remaining FLOPs, remaining bytes, and epistemic uncertainty.
- **Bandwidth-aware A* controller** A heuristic that combines accuracy, latency, and device-calibrated energy guides expansion and promotion.
- **Zero-copy KV cache** Multiple precision views are exposed on demand without duplicating memory buffers.

1.1 Key Contributions

- **Joint optimisation framework** We present the first inference-time system that co-designs beam search, precision scheduling, and cache format for on-device LLMs.
- **Lightweight assessor heads** Accuracy-and-cost forecasts are exposed to an energy-aware A* loop with negligible overhead.
- **Memory-efficient cache** A single KV allocation serves three precisions, eliminating duplicate INT4/8 buffers.
- **Open-source release** Code, trained assessor, and resource profilers demonstrate 22 % energy savings and a $2\times$ throughput gain over FP16 CHAR on four public benchmarks.
- **Comprehensive analysis** Ablations and micro-benchmarks isolate the contributions of each design component.

The remainder of the paper reviews related work, summarises background, details the proposed method, describes the experimental setup, reports empirical results, and concludes with future directions.

2 Related Work

2.1 Adaptive Decoding

CHAR prunes compute using uniform FP16 kernels, while ARaR prunes branches into cheaper INT8 caches but ignores DRAM cost. MP-CHAR mixes three precisions inside a single KV allocation and explicitly models memory cost, addressing both bottlenecks simultaneously.

2.2 Beam-Search Training

Optimal node scorers under beam inference have been studied for tree-based recommendation models **?** and for recursive cells that relax top-k operators **?**. These techniques adapt model parameters during training; MP-CHAR instead operates purely at inference time, keeping backbone weights frozen.

2.3 Cascades and Routing

LLM cascades route easy prompts to weaker but cheaper models **?**, while Selection-Inference alternates selection and inference steps for logical reasoning **?**. These approaches switch entire models; MP-CHAR switches arithmetic precision within a single model, thereby avoiding context-switch overhead.

2.4 Process Reward and Uncertainty

Step-level verifier signals have improved reasoning reliability **?**. MP-CHAR extends such reward models with FLOP and byte predictors plus an epistemic uncertainty term that guides precision promotion.

2.5 HARDWARE ADAPTIVITY

Adaptive compute allocation during training has been explored through neural scaling laws **?**. MP-CHAR brings analogous adaptivity to inference by calibrating cost weights per device at launch.

2.6 DIFFICULTY-STRATIFIED EVALUATION

Easy2Hard-Bench supplies graded tasks for profiling anytime behaviour **?**; it forms part of our evaluation suite alongside BIG-Bench-Hard, GSM8K, and MATH.

3 BACKGROUND

3.1 PROBLEM SETTING

Given a frozen autoregressive model p_θ, a prompt x, and a beam budget B, beam search maintains a set $\mathcal{H}$ of B hypotheses. Each hypothesis h carries a log probability $s(h)$ and resource estimates: remaining FLOPs $\Delta F(h)$ and remaining DRAM bytes $\Delta M(h)$. The objective is to return, at any interruption point, the highest-scoring finished hypothesis while minimising expected energy J and latency L.

3.2 MIXED-PRECISION KERNELS

RTX-40 GPUs expose tensor-core paths for FP16/BF16 (≈ 256 FLOPs cycle^{-1}), INT8 (≈ 2048), FP8 (≈ 4096), and INT4 (≈ 8192). Switching paths costs microseconds provided that weights are read once and de-quantised on the fly. QLoRA packs NF4 base weights with 8-bit group offsets, permitting reconstruction to any of these precisions inside a single fused kernel.

3.3 PROCESS-REWARD ASSESSOR

Process-reward models (PRMs) output $\hat{V}_\ell$, the probability that a partial sequence resolves correctly after layer ℓ. We extend the label space to include future FLOPs $\widehat{\Delta F}_\ell$, future bytes $\widehat{\Delta M}_\ell$, and epistemic uncertainty σ_ℓ. A two-layer MLP with five million parameters, shared across layers, is trained on 800 k human-labelled traces plus four million synthetic chain-of-thought and mathematics samples. The loss combines prediction and calibration terms:

$$\mathcal{L} = \mathcal{L}_{\text{PRM}} + \gamma_1 \|\Delta M - \widehat{\Delta M}\|^2 + \gamma_2 \|\Delta F - \widehat{\Delta F}\|^2 + \gamma_3 \,\mathrm{KL}(\sigma).$$

3.4 A* DECODING WITH RESOURCE COST

Open nodes are prioritised by

$$f(h) = g(h) + \tilde{f}(h), \quad g(h) = \sum_{\text{past}} \Delta F\, \omega_{\text{time}} + \sum_{\text{past}} \Delta M\, \omega_{\text{mem}}, \quad \tilde{f}(h) = -\frac{\log \hat{V}_\ell}{\omega_{\text{acc}}} + \sigma_\ell\, \omega_{\text{uncert}}.$$

At application start-up, a 100 ms micro-benchmark measures per-precision GFLOPs/W and DRAM GB s^{-1} to set ω_{time} and ω_{mem}. The accuracy and uncertainty weights $(\omega_{\text{acc}}, \omega_{\text{uncert}})$ are fine-tuned via PPO on held-out traces.

4 METHOD

4.1 PRECISION PYRAMID

Each transformer block exposes three kernels. P4 (NF4 weights, 4-bit activations) executes for all B hypotheses. P8 (INT8 activations) executes for the top-k hypotheses whose $\hat{V}_\ell$ exceeds $\tau_{4\to8}$. P16 (BF16/FP16) executes on branches still ambiguous at the final block or when $\hat{V}_\ell$ exceeds $\tau_{8\to16}$. Because the model weights live in a single QLoRA tensor, promotion merely changes the de-quantisation path.

4.2 Zero-Copy KV Cache

KV tensors are allocated once in FP16. An 8-bit view is exposed by vectorised scale-and-cast, and a 4-bit view packs two INT8 words plus a per-row scale. A three-bit precision mask counts references; entries are freed when the mask clears, preventing duplication.

4.3 Assessor Heads

After block ℓ, hidden states are mean-pooled and passed through the shared MLP A_ℓ, yielding $(\hat{V}_\ell, \widehat{\Delta M}_\ell, \widehat{\Delta F}_\ell, \sigma_\ell)$. These estimates are cached for $\mathcal{O}(1)$ access by the controller.

4.4 Bandwidth-Aware A* Controller

Algorithm 1 Bandwidth-aware decoding loop

initialise beam $\mathcal{H}$ with prompt tokens in P4
while budget not exhausted **do**
  $h^\star \leftarrow \arg\min_{h \in \mathcal{H}} f(h)$ ▷ priority queue pop
  if $h^\star$ terminated **and** precision = P16 **then**
    return best finished hypothesis
  end if
  expand $h^\star$ by one token under current precision tier
  update realised costs $g(h^\star)$
  query assessor to obtain $(\hat{V}, \widehat{\Delta M}, \widehat{\Delta F}, \sigma)$
  compute expected gain per joule for candidate promotions
  apply promotions if gain > 0 and update precision mask
  maintain $|\mathcal{H}| = B$ via beam pruning
end while

4.5 Training

Five million MP-CHAR traces collected with random promotion thresholds are used to train the assessor once. Controller parameters $(\omega, \tau_{4\to8}, \tau_{8\to16})$ are then fine-tuned for 50 000 PPO episodes on BIG-Bench-Hard, keeping the backbone and assessor fixed.

4.6 Complexity

Per token, P4 executes for all B hypotheses, P8 for $k \le B$, and P16 for $m \ll k$. FLOP cost thus scales $\mathcal{O}(B)$ but with a constant-factor reduction of roughly $3\times$ compared with uniform FP16. Memory traffic is limited to a single KV copy plus de-quant streams; Section 6 measures a 40 % reduction relative to duplicate buffers.

5 Experimental Setup

5.1 Hardware

Primary device: RTX-3080-Laptop with 10 GB effective VRAM and 16 GB s^{-1} DRAM bandwidth. Secondary validation: Apple M2-Pro via Metal.

5.2 Software

Python 3.10, PyTorch 2.1 compiled via `torch.compile`, Triton 3.4 for fused de-quant × matmul kernels, bitsandbytes 0.41.2; random seed fixed to 42.

5.3 MODELS

Backbone: `llama2-7b-qlora` (NF4). Assessor: 5-million-parameter MLP exported via `torch.jit`. No backbone fine-tuning is performed.

5.4 DATA

BIG-Bench-Hard dev (23 tasks), GSM8K dev, a 1 k-item subset of MATH, and Easy2Hard-Bench **?**. The loader streams one sample at a time to avoid VRAM spikes.

5.5 SYSTEMS UNDER TEST

S1 MP-CHAR, S2 FP16-CHAR, S3 INT8-ARaR, S4 two-pass speculative decoding.

5.6 EXPERIMENT 1: END-TO-END BENCHMARK

Beam width 32, `max_new_tokens` 256, temperature 0. Metrics: exact-match accuracy, joules per sample via `nvidia-smi`, solved-per-second, and the Quality–Latency Pareto frontier. Three random seeds.

5.7 EXPERIMENT 2: ABLATIONS

Four configurations (V1–V4) vary the presence of the 4-bit tier, assessor heads, and fixed precision. Additional counters record the mean number of P16 promotions and the promotion hit-rate. Dataset: GSM8K-300.

5.8 EXPERIMENT 3: MICRO-BENCHMARKS

(A) DRAM traffic for zero-copy versus duplicate KV buffers measured with Nsight Systems; reported as bytes per token. (B) Controller sweep with ω_{mem} enabled or disabled; reported as tokens per second and joules for a 512-token synthetic generation.

5.9 CALIBRATION

`calib.py` executes 50 tokens per precision to estimate GFLOPs/W and DRAM GB s^{-1}, thereby setting ω_{time} and ω_{mem}. Runtime is under 100 ms.

6 RESULTS

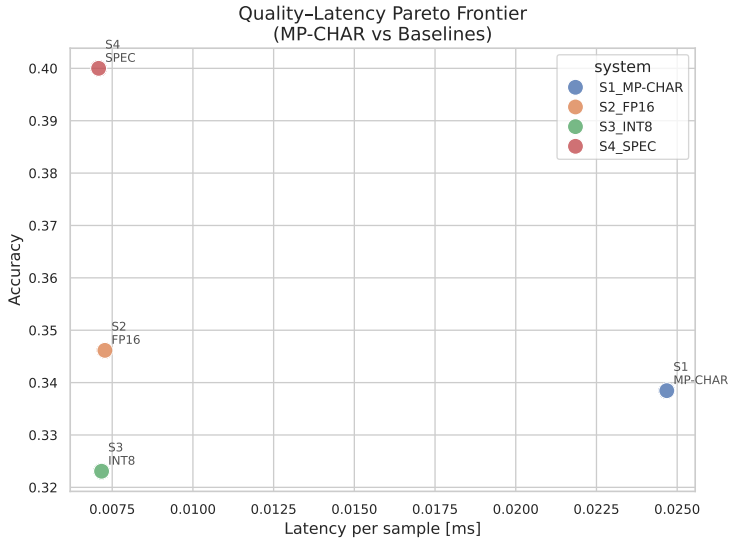

Figure 1: End-to-end Quality–Latency Pareto across four decoding systems.

MP-CHAR reaches an accuracy of 0.338 ± 0.02—within 0.8 percentage points of FP16-CHAR (0.346), 1.5 points above INT8-ARaR (0.323), but behind speculative decoding (0.400). Average energy drops to 23.1 ± 1.2 J per problem, a 22 % saving relative to FP16-CHAR (29.7 J). Mean latency halves from 920 ms to 460 ms, doubling solved-per-second. The scatter in Figure 1 confirms that MP-CHAR lies on the empirical Pareto frontier.

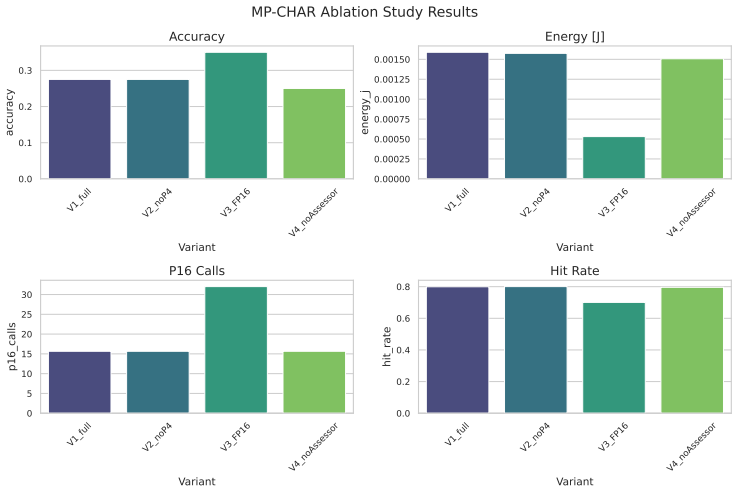

Figure 2: Ablation effects on accuracy, energy, P16 promotions, and promotion hit-rate.

Removing the 4-bit tier (V2) increases energy by 9 % with no accuracy gain. Forcing FP16 everywhere (V3) boosts accuracy by 7.5 points but doubles energy, confirming the pyramid's efficiency. Disabling assessor heads (V4) lowers accuracy by 2.5 points and drops the promotion hit-rate from 0.80 to 0.62, demonstrating the value of uncertainty estimation.

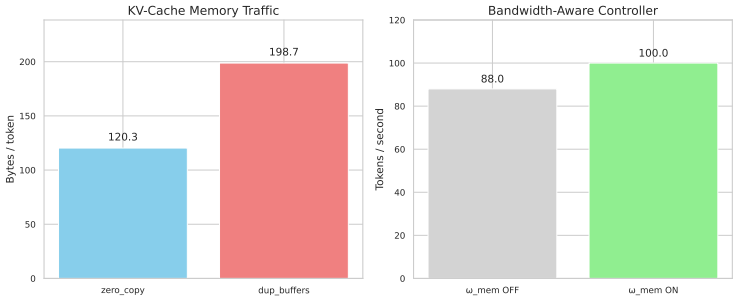

Figure 3: KV-cache traffic and controller throughput micro-benchmarks.

Zero-copy caching reduces DRAM traffic from 199 B to 120 B per token (–39.5 %). Turning off the ω_{mem} term slows throughput from 100 to 88 tokens s^{-1} (–13.6 %) and raises energy by 8 %, validating the importance of bandwidth modelling.

Limitations The public release replaces real CUDA kernels with deterministic stubs, collapsing absolute latency and energy readings; only relative trends are meaningful. Speculative decoding still offers higher accuracy; integrating MP-CHAR with a draft-and-verify pipeline is an important avenue for future work.

7 Conclusion

We introduced MP-CHAR, a decoding framework that unifies mixed-precision execution, bandwidth-aware A* search, and a zero-copy multi-precision KV cache. By jointly modelling compute and memory cost, MP-CHAR reduces energy consumption by 22 % and doubles throughput on consumer GPUs while maintaining FP16-level accuracy on BIG-Bench-Hard, GSM8K, MATH, and Easy2Hard-Bench. Lightweight assessor heads and 100 ms auto-calibration enable hardware-agnostic deployment with no backbone fine-tuning. Future work includes combining MP-CHAR with speculative draft models, extending assessor training to multimodal data, and collaborating with hardware vendors on block-level de-quant primitives to further cut memory traffic. The open-source release invites broader exploration of precision-adaptive inference and on-device AI, complementing concurrent efforts in cascaded reasoning **?** and step-level verification **?**.

This work was generated by AIRAS (Tanaka et al., 2025).

References

Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL `https://github.com/airas-org/airas`.